

λ -kalkylen

Hans

September 2025

λ -kalkylen er beregningsmodellen, der ligger til grund for Haskell og andre funktionelle programmeringssprog. I denne korte note vil jeg skitsere grundlæggende elementer af λ -kalkylen.

λ -kalkylen er et lille sprog, der indeholder de sproglige konstruktioner, som ethvert funktionelt programmeringssprog skal have: Evnen til at definere funktioner og evnen til at kalde en funktion med et argument. Ethvert funktionelt programmeringssprog indeholder en version af λ -kalkylen.

Syntaks Lad os først betragte den rene λ -kalkyle, der ikke har nogen foruddefinerede konstanter eller nogen form for typer. Vi betragter en version, hvor udtryk har den abstrakte syntaks givet ved opbygningsreglerne:

$$e ::= \underbrace{x}_{\text{variable}} \mid \underbrace{\lambda x.e_1}_{\text{abstraktion}} \mid \underbrace{e_1 e_2}_{\text{applikation}}$$

I denne version af sproget er der ingen tal, sandhedsværdier eller andet (det kan man sagtens indføre, men det gør vi ikke nu). Vi kan stadig definere interessante funktioner.

Eksempel 1. Identitetsfunktionen kan skrives som $\lambda x.x$.

Vi kan bruge parenteser, når det er nødvendigt. Udtrykket $\lambda x.e_1$ betegner en funktion med den formelle parameter x og krop e_1 . Scope for x er **hele** e_1 . Vi kan indføre parenteser for at afgrænse scope, hvis det er nødvendigt. Udtrykket $e_1 e_2$ betegner, at funktionen e_1 skal anvendes på argumentet e_2 .

Scoperegler I en abstraktion $\lambda x.e$ kaldes x for en *bindende forekomst* af x . Hvis den samme x forekommer inde i e , kalder vi det en *bundet forekomst* af x . En bindende forekomst skal opfattes som en pladsholder; dens navn er ikke vigtigt. For eksempel er både $\lambda x.x$ og $\lambda y.y$ udtryk, der beskriver identitetsfunktionen. I $\lambda x.x$ er den første x den bindende forekomst (der kan højst være én bindende forekomst af en given variabel i en abstraktion), og den anden er en bundet forekomst (der kan være mange sådanne bundne forekomster). For at undgå at komplicere semantikken og begrebet substitution, *antager vi altid, at alle bindende forekomster bruger forskellige variabelnavne*. Hvis det er nødvendigt, omdøber vi systematisk de bindende forekomster og de bundne navne. Hvis et udtryk e_2 kan opnås fra et andet udtryk e_1 ved systematisk at omdøbe nul eller flere bindende forekomster og deres bundne forekomster, siger vi, at e_1 og e_2 er α -konvertible og skriver $e_1 \equiv_\alpha e_2$.

Eksempel 2. $\lambda x.x$ kan α -konverteres ved at omdøbe x til y . Vi har $\lambda x.x \equiv_\alpha \lambda y.y$.

I $\lambda x.xy$ er y ikke bundet, så vi må ikke omdøbe x til y , for så forveksler vi en fri variabel med en bunden. Men vi må gerne omdøbe x til z . Så vi har $\lambda x.xy \equiv_\alpha \lambda z.zy$, men $\lambda x.xy \not\equiv_\alpha \lambda y.yy$.

Semantik Vi kan give λ -kalkylen en small-step-semantik, hvor overgange har formen:

$$e \rightarrow e'$$

Vi skal læse dette som at udtrykket e reducerer i ét skridt til udtrykket e' . Transitionsrelationen $\rightarrow$ kalder vi her for *reduktionsrelationen*.

β -reglen beskriver, hvordan vi evaluerer en applikation $e_1 e_2$, når e_1 er en abstraktion:

$$(\text{BETA}) \quad (\lambda x.e_1)e_2 \rightarrow e_1[x \mapsto e_2]$$

Notationen $e_1[x \mapsto e_2]$ betyder, at den faktiske parameter e_2 substitueres for hver forekomst af den formelle parameter i kroppen af e_1 . Så denne regel beskriver funktionskald.

Vi kalder et underudtryk, for hvilket en reduktion er mulig, for et *redex*; hvis reduktionen er en β -reduktion, taler vi om et β -redex.

De andre reduktionsregler beskriver, hvordan en applikation vil opføre sig, hvis vi ikke udfører en beta-reduktion, og at α -konvertible udtryk har de samme reduktioner.

$$\begin{array}{ll}
\text{(VENSTRE)} & \frac{e_1 \rightarrow e'_1}{e_1 e_2 \rightarrow e'_1 e_2} \\
\text{(HØJRE)} & \frac{e_2 \rightarrow e'_2}{e_1 e_2 \rightarrow e_1 e'_2} \\
\text{(ALFA)} & \frac{e_1 \equiv_\alpha e_2 \quad e'_1 \equiv_\alpha e'_2 \quad e_2 \rightarrow e'_2}{e_1 \rightarrow e'_1}
\end{array}$$

Vi kan altså udføre reduktion i ethvert underudtryk af en applikation, men *ikke* under en abstraktion, da der ikke er regler, der skriver at man skal kunne det.

Eksempel 3. Her er et eksempel på en reduktion:

$$\begin{aligned}
(\lambda x. \lambda y. yx) \underbrace{((\lambda z. zz)y)}_{\text{redex}} &\rightarrow (\lambda x. \lambda y. yx)(yy) \\
&\equiv_\alpha (\lambda x. \lambda w. wx) \underbrace{(yy)}_{\text{redex}} \\
&\rightarrow \lambda w. wyy
\end{aligned}$$

Hvis vi ikke havde α -konvertering, ville vi støde på problemet med navnesammenstød, da der er to forskellige y 'er i det oprindelige udtryk — et ene er bundet, det andet er frit.